

Video Game Programming: Advanced Game Rendering

L15

Prof. Fabio Buttussi, PhD
Human-Computer Interaction Laboratory
Department of Mathematics, Computer Science, and Physics
University of Udine - Italy

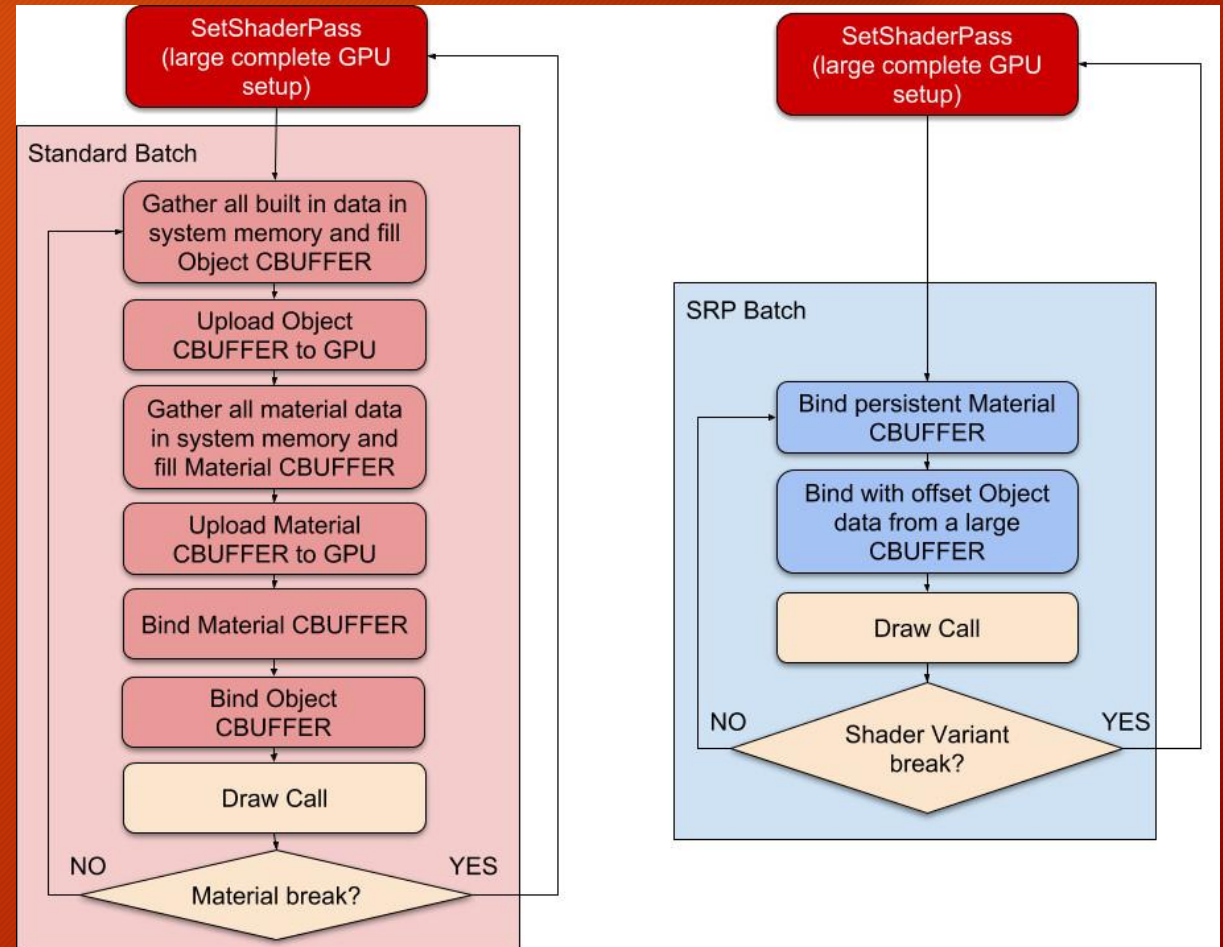
Scriptable Render Pipelines

- Scriptable RPs are Unity RPs that can be scripted using C#
- Universal RP and High-Definition RP are pre-configured SRPs
- Game programmers can also build their custom RPs based on SPR
- Every SPR has:
 - An RP asset with configuration settings
 - An RP instance with overridable events



Batchers

- Batchers are a rendering loop to speed up rendering
- Standard batcher is useful when several meshes use the same material
- SRP batcher is useful when several materials use the same shader
- SRP batcher can be used for Lit/Unlit URP/HDRP shaders



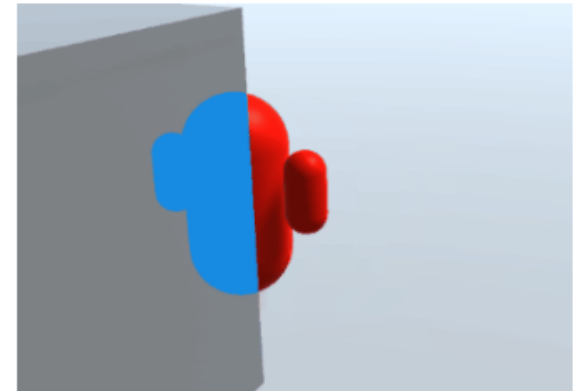
Universal Render Pipeline

- URP is an SRP to optimize graphics across platforms
- It has some minor limitations and several advantages over built-in RP (<https://docs.unity3d.com/Manual/render-pipelines-feature-comparison.html>)
- It is highly customizable using URP asset settings
- Different URP assets can be used for different quality settings and platforms



URP render features

- Renderer features are assets to add extra render passes to an URP forward renderer and configure their behavior
- Settings allow to specify when features are applied in the rendering cycle and filtering objects on which they are applied
- Render features can change the material for filtered objects in particular conditions (in fig.)
- A special render feature provides Screen Space Ambient Occlusion (in fig.) that darkens holes and surfaces close to each other, because they occlude ambient light



URP lights

- URP has some specific light settings in the inspector
- They can be accessed in code by using Universal Additional Light Data component, which is automatically attached to all lights
- URP has limited support for real-time global illumination
- URP baked global illumination supports three modes for mixed lights (Baked Indirect, Subtractive, and Shadowmask)

URP camera

- URP has new settings (and an additional component for code access) also for cameras
- Render Type setting allows to have base and overlay cameras
- In URP, camera stacking allows to combine different camera layers
- E.g., camera stacking is useful to have 3D models in a 2D UI (in fig.)
- Cameras can trigger post-effects when they enter defined volumes

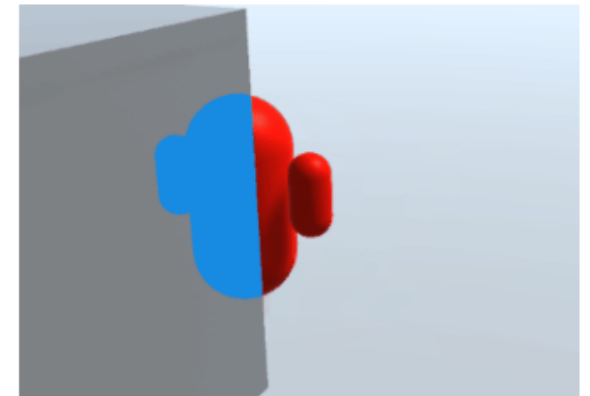


URP volumes and post-effects

- Global volumes store effects that are applied to whole scene
- Local volumes have trigger colliders that define portions of the scene where specific effects are applied (e.g., in a tunnel)
- URP has several pre-defined post-effects, such as:
 - Vignette, which mimics loss of light in the corner of camera frame
 - Bloom, which simulates strong light bleeding on camera sensor
 - Motion Blur, which simulates blurring when objects move fast
 - Tonemapping, which remaps HDR values to a new range
 - Depth of Field, which simulates the focus properties of a camera with some sharply focused objects and others out of focus
- Some are mobile friendly, and others are not

URP shaders

- Standard shader does not work with URP
- URP has its own set of shaders:
 - Lit shader handles lights and is physically-based
 - Unlit shader did not handle lights
 - Simple Lit shader handles lights, but is not physically-based
 - Complex Lit has more features
 - Baked Lit handles only baked lights
 - There are also specific shaders for particles



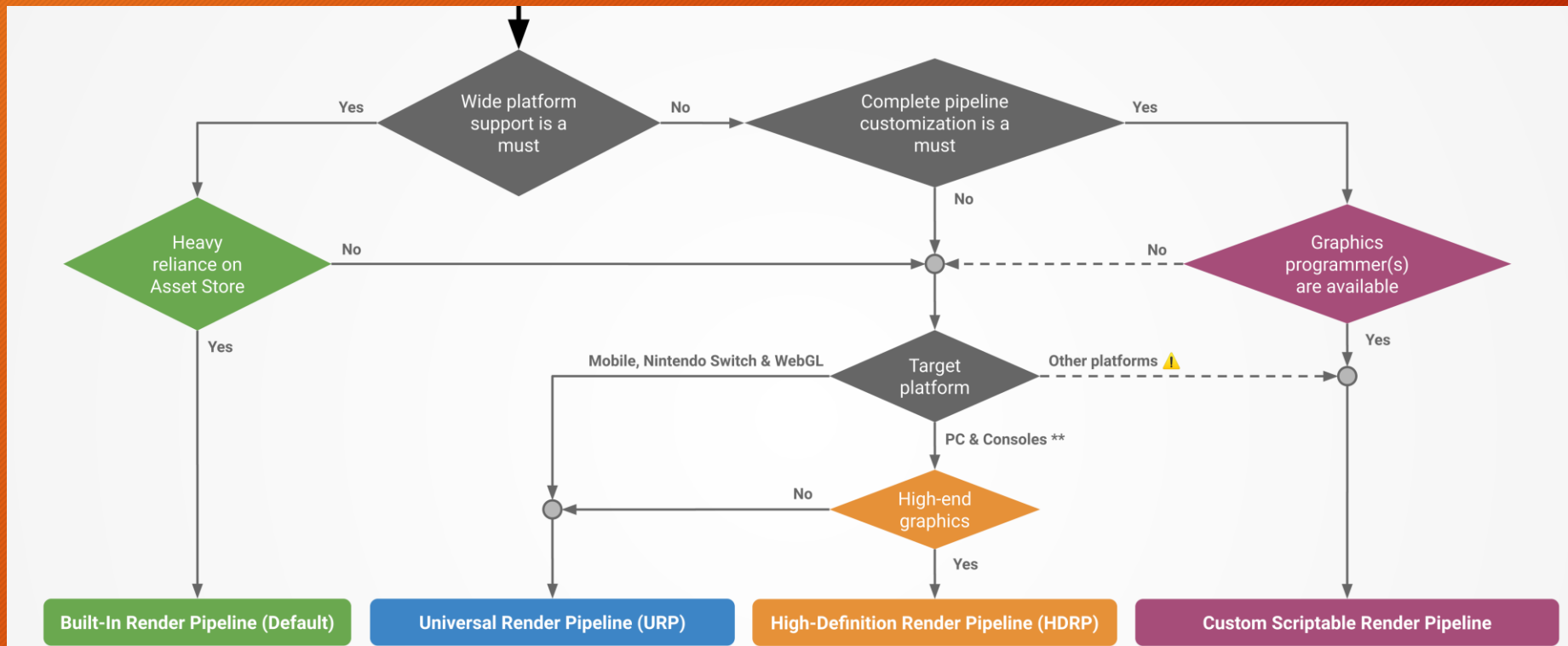
Example with Unlit
and Lit shader

High-Definition Render Pipeline

- The HDRP offers several additional features:
 - Translucency and iridescence
 - Shaders for hair and tissues
 - New light types
 - Volume-based sky and fog
 - Ray tracing
 - New Realtime Global Illumination
 - ...
- It works only on PCs and some consoles



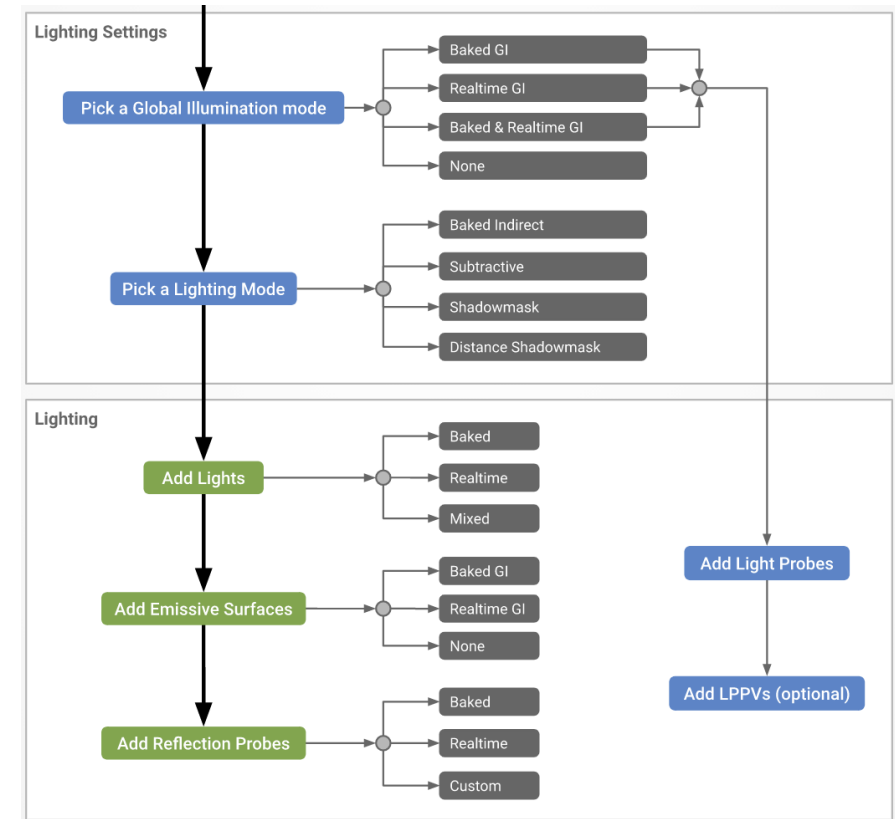
Render pipeline choice



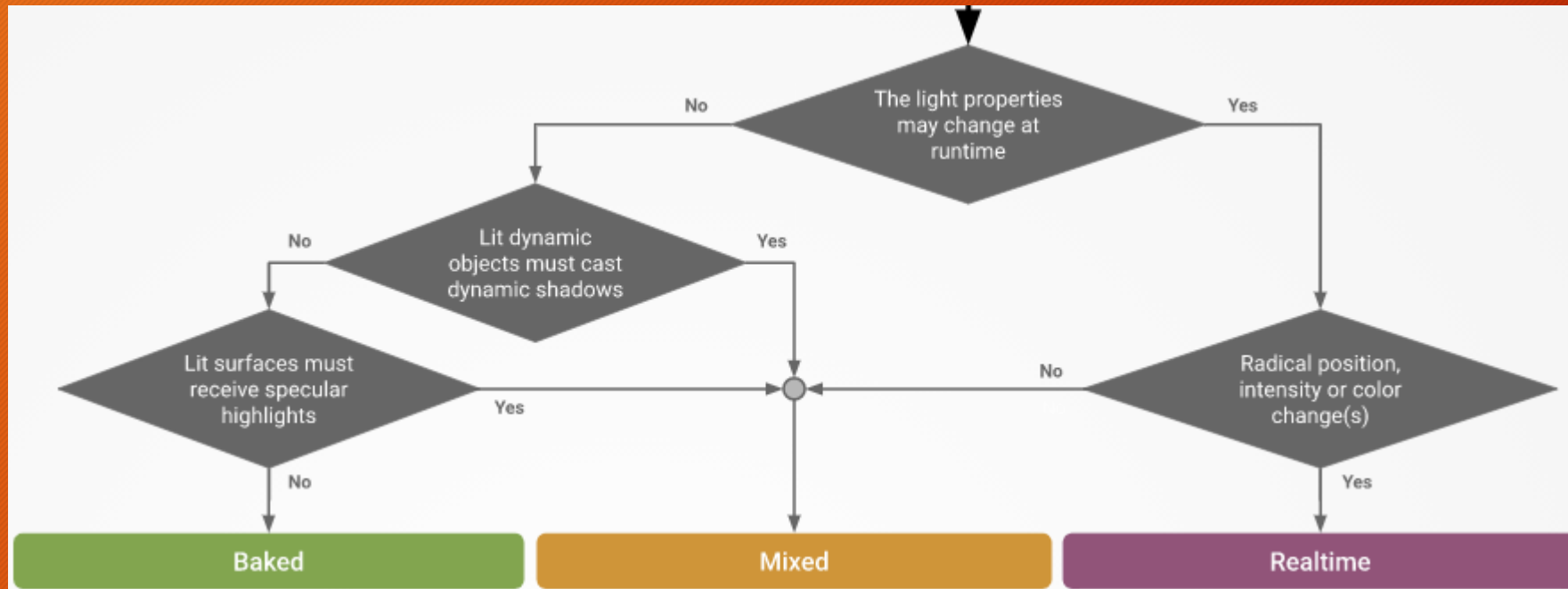
Deprecated from Unity 6.5,
supported until Unity 6.7 LTS

Global Illumination

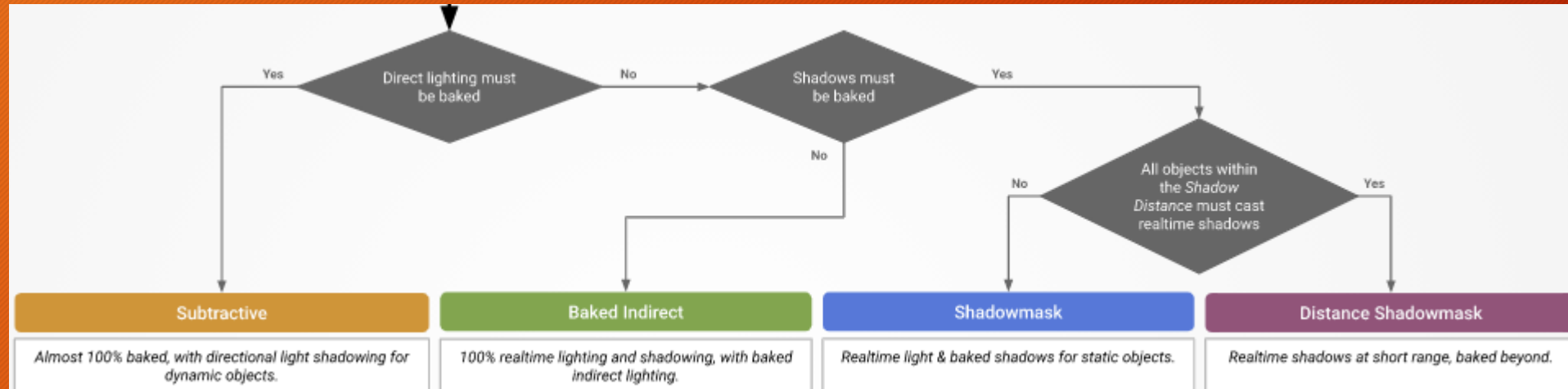
- To increase lighting quality, game engines implement Global Illumination (GI), which considers both direct and indirect lights
- In Unity, GI can be real-time (limited support) or baked (supported by all RPs)
- According to light types and lighting mode, some lights are baked and others are calculated in real-time
- Baked data are stored in lightmaps, light probes, and reflection probes
- GI static flag and generate UV lightmaps of imported models should be checked



Light type choice

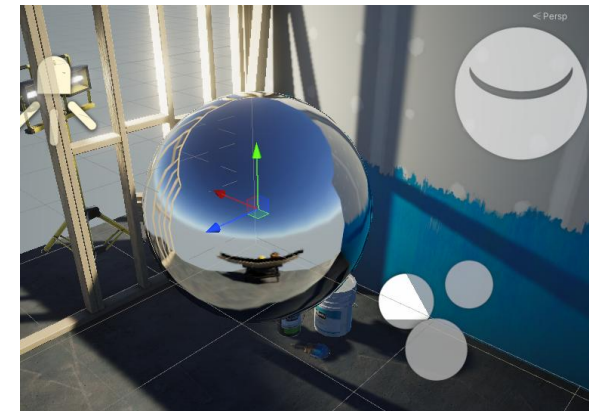
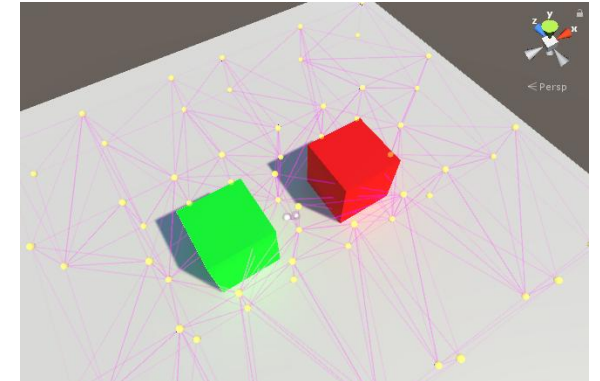


Lighting mode choice



Probes

- Light probes provide a way to capture and use information about light that is passing through the empty space in the scene
- Light information stored in probes is mainly used to illuminate moving objects with baked lights
- Reflection probes sample the environment at strategic scene points using cubemaps
- Reflection information stored in probes is mainly used to show reflections on metals



High Dynamic Range

- HDR allows the brightest part of the image to be greater than 1
- This gives a wider range of light intensities, more realistic colors, and less saturation, even with bright light
- To display the output on a non-HDR device a tonemapping post-effect is required
- Figures show an HDR scene without and with tonemapping

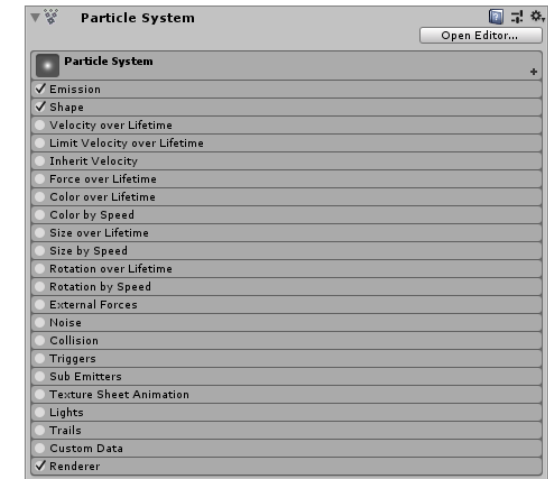


Shader creation

- Unity supports the creation of custom shaders in different ways:
 - Writing a shader program in OpenGL Shading Language
(<https://docs.unity3d.com/Manual/SL-GLSLShaderPrograms.html>)
 - Using a shader programming language called HLSL
(<https://docs.unity3d.com/Manual/SL-ShaderPrograms.html>)
 - Using a Unity-specific language called ShaderLab
(<https://docs.unity3d.com/Manual/SL-Reference.html>)
 - Using Shader Graph, a visual tool to build URP and HDRP shaders
(<https://docs.unity3d.com/Packages/com.unity.shadergraph@17.6/manual/index.html>)

Particle systems

- Particle systems simulate several small images or meshes (particles) to produce a visual effect
- Each particle represents an individual graphical element
- Unity has two particle systems:
 - Built-in for all RPs (in fig.)
 - Visual Effect Graph for HDRP and URP (<https://docs.unity3d.com/Packages/com.unity.visualeffectgraph@17.6/manual/index.html>)
- Built-in particle system has several options that can also be controlled by code



Bibliography

- Unity, Lighting up your graphics, 2020, <https://create.unity3d.com/real-time-lighting-for-games-in-unity>
- Gomila Salas Juan Gabriel, Borromeo Nicolas Alejandro, Hands-On Unity Game Development, 4th ed., Packt Publishing, 2024
- Dinur, The Complete Guide to Photorealism: For Visual Effects, Visualization and Games, Routledge, 2022

